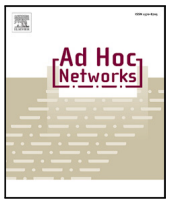
Ad Hoc Networks 178 (2025) 103936



Contents lists available at ScienceDirect

Ad Hoc Networks

journal homepage: www.elsevier.com/locate/adhoc





Computation offloading in MEC-assisted vehicular networks with task migration and result feedback

Jingwei Geng, Shunfu Jin *

School of Information Science and Engineering, Yanshan University, Qinhuangdao, China

ARTICLE INFO

Keywords:
Computation offloading
Task migration
Result feedback
Edge computing
Lyapunov optimization

ABSTRACT

In vehicular networks, roadside units (RSUs) with mobile edge computing (MEC) assistance bring computing resources to the edge for enhancing the computation capacities of vehicles. However, uneven distribution of vehicles leads to load imbalance between MEC computing servers and this poses a huge challenge to computation offloading in vehicular networks. In this paper, we consider task migration between RSUs with different loads on a horizontal scale and computation result feedback in a MEC-assisted computation offloading scenario. We develop a vehicle trajectory prediction module based on deep neural networks for predicting the vehicle position after a task is completed and calculating the delay and energy consumption in the result feedback process. We formulate a computation offloading problem with the aim of minimizing total computation cost within continuous time slots. To address the coupling of decisions under different time slots, we propose a Lyapunov-based novel online heuristic offloading (LNOHO) algorithm for the formulated problem. Applying the Lyapunov optimization framework, the original multi-slot problem is decomposed into multiple per-slot subproblems. Each subproblem is a nonlinear integer programming (NIP) problem. For such an NP-hard problem, we divide it into three processes and propose a load-aware migration heuristic (LMH) algorithm with low complexity to obtain per-slot decisions. The simulation results based on real road topology show that our proposed vehicle trajectory prediction module and algorithm can achieve better performance.

1. Introduction

Vehicular networks are a technology that connects vehicles to the Internet, enabling information exchange and sharing between vehicles through the vehicle-to-infrastructure (V2I) link. The emergence of vehicular networks has promoted the intelligence level of vehicles, while also giving rise to numerous computation-intensive and delay-sensitive applications, such as smart traffic control [1], high definition (HD) map updates [2], and image or video-assisted real-time navigation [3], etc. These tasks demand substantial computation resources while requiring completion within tight time constraints. Although vehicles are equipped with local servers, the limited computation power makes it challenging to efficiently process the above tasks. Taking HD map updates tasks as an example, local computing in complex urban environments may result in processing latency over 50 ms (exceeding the 30 ms threshold required by the decision-making module) [4].

Considering that cloud computing can provide rich computing resources [5], cloud-assisted vehicle networks are proposed to provide computing offloading for vehicle tasks. Due to the fact that cloud networks are typically deployed far away from vehicles, vehicles may need to endure high propagation delays when offloading tasks to the cloud [6]. As an extension of cloud computing, mobile edge computing (MEC) avoids the defects of high propagation delays in cloud networks by deploying computing servers in roadside units (RSUs) close to vehicles, thus providing low latency and low energy consumption services to vehicles [7,8]. In MEC-assisted vehicular networks, vehicles can choose to process locally or offload to nearby RSUs. However, when RSUs are deployed in traffic-intensive areas, the high vehicle density may trigger excessive computation demands. RSUs with limited computing capacity might fail to handle concurrent task requests from excessive vehicles. Based on the vehicular trajectory dataset in Shenzhen, the authors in [9] visualized the RSU load distribution, demonstrating that RSUs are overloaded in urban agglomeration areas. In order to ensure that the system can operate properly in this case, RSUs always resort to service request filtering mechanisms, which inevitably compromise QoS. Given that computation workloads of RSUs follow the first law of geography, peripheral RSUs far from urban cores exhibit greater resource availability. By implementing RSU-RSU collaboration through migrating tasks from RSUs with higher load levels to RSUs with lower load levels, the network gains enhanced capability to

* Corresponding author.
E-mail address: jsf@ysu.edu.cn (S. Jin).

https://doi.org/10.1016/j.adhoc.2025.103936
Received 4 March 2025; Received in revised form 22 May 2025; Accepted 25 May 2025
Available online 23 June 2025
1570-8705/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

accommodate traffic bursts, thereby improving computation resource utilization efficiency.

For this, some researchers investigated the task migration mechanisms in vehicular networks [10–16]. Migrating tasks from RSUs with higher load levels to RSUs with lower load levels can enhance the collaboration between RSUs, enabling the network to better respond to sudden traffic situations and thereby improve the utilization of computing resources. In the above literature, the authors assume the data size of a task computation result is smaller, and the result feedback delay is negligible. For vehicle tasks involving VR and mobile online gaming, due to the large data size of the computation results generated by these tasks, such an assumption does not hold. Taking VR gaming as an example, although users can only perceive a small portion of the image within their field of view, the information processed by RSUs contains comprehensive viewpoint information, resulting in substantial redundancy in computation results [17]. Furthermore, due to task migration and vehicle mobility, computation result may undergo RSU handovers, introducing additional delay. Therefore, it is necessary to consider the result feedback delay when making offloading decisions. If the vehicle has already moved out of the original RSU's communication range, the computation results need to be sent to the nearest RSU of the vehicle through the infrastructure-to-infrastructure (I2I) link, and then sent to the vehicle through the infrastructure-to-vehicle (I2V) link. Although some researchers explored the result feedback in computation offloading [18–21], there are still the following defects: (1) the cross-regional mobility of the vehicle is ignored during task offloading; (2) due to the uncertainty of vehicle mobility, it is difficult to obtain in advance the position of the vehicle when a task is completed. Thus, the delay and energy consumption of tasks in the result feedback process are difficult to compute.

In order to cope with computation offloading in MEC-assisted vehicular networks with task migration and result feedback, we face the following challenges:

- In a real scenario, considering the uneven distribution of traffic flow in time and space and the difficulty in obtaining future traffic flow information of RSUs, how to make an online offloading decision to ensure that the load difference of RSUs does not increase over time.
- The position of a vehicle after a task is completed is future information before a decision is made. For this, a vehicle trajectory prediction module needs to be designed. Considering that the trajectory of a vehicle is not only related to its own motion state, but also related to surrounding vehicles and environmental information, non interactive prediction methods based on kinematics are no longer applicable. Therefore, it is crucial to design a novel data-driven prediction method using artificial intelligence (AI) methods.
- To minimize the total computation cost (weighted sum of total delay and total energy consumption) of the overall vehicular networks, a nonlinear integer programming (NIP) problem regarding binary offloading decisions which has been proven to be NP-hard is usually involved. As the problem scale increases, the size of the solution space increases exponentially, therefore, designing an optimization algorithm with low-complexity to obtain a high-quality solution in a short time is necessary.

To fill the gaps and address the challenges in MEC-assisted vehicular networks architecture, we investigate computation offloading with task migration and result feedback. We comprehensively consider total delay and total energy consumption, and formulate a computation offloading problem that minimizes total computation cost within continuous time slots. We design a vehicle trajectory prediction module based on deep neural networks to predict vehicle position. Considering that RSUs cannot obtain traffic flow information for future time slots, we develop an online optimization algorithm to solve the formulated problem. The main contributions of this paper are described as follows:

- We build a MEC-assisted vehicle network, in which task migration and result feedback are considered simultaneously. To ensure the long-term stability of the system, we formulate a computation offloading problem with a long-term objective function and a long-term constraint.
- We develop a deep neural network model based on an encoder–decoder architecture to predict vehicle trajectories. The encoder structure consists of a graph convolutional neural (GCN) network and a long short-term memory (LSTM) network. The GCN network is used to extract the surrounding environment information of the vehicle, and the LSTM network is used to extract the historical motion information of the vehicle. The decoder structure has a LSTM-self attention network, which predicts vehicle trajectories by decoding the information output by the encoder.
- To address the formulated problem, we propose a Lyapunov-based novel online heuristic offloading (LNOHO) algorithm. Specifically, with the help of the Lyapunov optimization framework, the continuous time slot NIP problem is decoupled into multiple NIP subproblems for each deterministic time slot. We propose a load-aware migration heuristic (LMH) algorithm with low-complexity to make a set of high-quality solutions for each time slot.
- By adjusting key parameters such as number of vehicles, we conduct extensive comparative experiments in a large-scale traffic scenario based on a real-world road topology, with 12km^2 area and 70 RSUs. The experimental results demonstrate that our designed vehicle trajectory module has high accuracy in predicting the position of the target vehicle. In addition, our proposed LNOHO algorithm achieves optimal performance compared to Lyapunov ant colony algorithm, Lyapunov quantum behavior particle swarm optimization algorithm, Lyapunov genetic with directed search algorithm, and Lyapunov random algorithm.

The remainder of the paper is organized as follows. In Section 2, we review related work. In Section 3, we describe system model and formulate the optimization problem. In Section 4, we design LNOHO algorithm and trajectory prediction module. In Section 5, we discuss experiment results. Section 6 concludes this paper.

2. Related work

Compared with cloud computing, edge computing is beneficial in reducing propagate delay, there are extensive research focusing on the task offloading in MEC systems. Ding et al. [22] jointly optimized power and time allocation to reduce the energy consumption of computation offloading in non-orthogonal multiple access (NOMA)-assisted MEC. To reduce the processing pressure on high-load MEC servers and ensure QoS, many researchers tried to explore computation offloading with task migration. Zhang et al. [10] considered a multi-layer network architecture, where tasks at the edge layer can be migrated to the cloud to receive services when they cannot be processed on time. Huang et al. [11] considered a cloud-side collaboration problem in a dynamic network scenario. Based on this, they proposed a joint computation offloading and resource allocation scheme. However, the above studies focus mainly on vertical task migration between edge layer and cloud layer. Migrating tasks from edge layer to cloud layer leads to offloading uncertainty due to the distance between cloud and edge layer. For this, some researchers started to study horizontal migration between edge servers. Dai et al. [12] investigated a new computation offloading scheme in MEC-assisted vehicular networks. By applying queue theory to model the task upload, migration, and computation processes, they proposed a collaborative computation offloading problem to minimize the delay in task completion. In order to strengthen the cooperation between edge servers in MEC-assisted vehicular networks, Liu et al. [13] developed an optimal task and resource scheduling scheme. This scheme fully explores the available vehicle resources and considers

task migration. Peng et al. [14] proposed a computation and communication cost-aware task migration scheme in MEC-assisted vehicular networks. On this basis, they formulated a migration optimization problem to maximize the service delay satisfaction and proposed a fast transfer reinforcement learning (RL) algorithm to obtain the optimal migration scheme. However, in these literatures [12–14], load balancing is ignored, therefore, the number of migrated tasks cannot be controlled, leading to the underutilization of computation resources.

Sahoo et al. [15] achieved load balancing by migrating virtual machines from overloaded RSUs to RSUs with spare resources. To reduce the migration cost, they proposed an application-aware load balancing algorithm. However, they ignore the variation of system energy consumption with load. Duan et al. [16] proposed a hierarchical management model under intelligent in-vehicle networks. Then, they developed the migration load minimization delay scheme. The load balancing problem of MECS is solved by choosing appropriate response time thresholds.

In [10–16], authors only considered the uploading, processing, and migration of tasks but ignored the result feedback. To improve offloading performance for adjacent vehicles, Sun et al. [19] designed an adaptive learning offloading algorithm to minimize the unloading delays. Chen et al. [20] studied a service configuration problem and proposed a multi-armed bandit learning algorithm to optimize feedback delay caused by the task transmission/computation. However, in [19, 20], the authors did not take into account the cross-regional mobility of vehicles. Nan et al. [21] studied a joint offloading and resource allocation problem with result feedback delay. Then, they designed separate algorithms according to the size of the scenario. However, the authors only considered the case where the computation is routed once. Wang et al. [18] considered two modes of migration of results, I2I and I2V, to accept the computation results. Then, they proposed task migration algorithms to minimize the computation cost. However, in [18], the movement of vehicles is based on the assumption that the vehicles are distributed according to a Poisson distribution on the highway. Such an assumption is obviously not consistent with practical scenarios. Moreover, although some studies investigated task migration and result feedback, there are few studies that consider both.

There are also extensive researches focusing on the optimization algorithms in vehicular networks computation offloading. To reduce the system latency, Yan et al. [23] proposed a LSTM network with attention mechanism and a deep deterministic policy gradient (DDPG) algorithm to jointly optimize unmanned aerial vehicle trajectories and offloading decisions. Wakgra et al. [24] formulated a multi-objective optimization problem to minimize delay and energy. Then, they proposed a distribution-based twin delayed deep deterministic policy gradient (TD3) to obtain the optimal offloading decision. Fang et al. [25] proposed a multi-agent twin delayed TD3 algorithm in reconfigurable intelligence surface (RIS)-aided MEC systems, which achieves maximization of secure offloading capability and minimization of energy consumption under different delay requirements of users. To solve the cooperative offloading problem for diverse tasks, Cui et al. [26] proposed an offline centralized training distributed execution framework based on improved soft actor critical. Hou et al. [27] proposed a distributed task offloading scheme based on multi-agent deep reinforcement learning (MADRL). In addition, to address the inefficiency caused by multi-agent credit allocation, they provided a counterfactual multi-agent reinforcement learning method to evaluate the behavior of each agent. The above RL algorithms generally apply to solving problems with continuous action spaces. However, the problem formulated in this paper is a NIP whose action space is discrete. Applying the above reinforcement learning algorithms to solve problems with discrete spaces requires that the continuous values output by the action network be converted to discrete values according to a set threshold. However, the optimization variables of the formulated NIP problem mentioned in this article are the NO. of the RSUs. For such a problem, it is difficult to apply a reasonable method to convert the continuous values output by the action network into discrete values. Therefore, the RL algorithms based on continuous control are unsuitable for solving the formulated problem in this paper.

Karimi et al. [28] proposed a collaboration between MEC and cloud decisions for different application offloading and then formulated a resource allocation problem to minimize response time. To solve such a NP-hard problem, they proposed deep Q-learning algorithm. Applying the above reinforcement learning algorithm to solve the problem requires that the action space is manageable. This is because too large an action space leads to an excessive number of neurons, which has the following drawbacks: (1) the requirements for the computation power and memory size of the GPU, and RAM size are substantially higher, which is difficult to be met by a general workstation; (2) the requirements for accuracy are substantially higher, which makes it difficult for the algorithm to converge. The action space of the formulated problem in this paper is much higher than the number of action spaces in existing literatures. Therefore, a discrete control based RL algorithm is not suitable for the formulated problem in this paper.

Wu et al. [29] proposed an algorithm based on the student admission model, which obtains an optimal solution by converting an integer planning problem into a many-to-one matching problem. Liwang et al. [30] developed an efficient unilateral-matching-based optimization algorithm that achieves optimal solutions with polynomial computational complexity. However, the above optimization algorithm is only suitable for solving an integer linear programming problem (ILP), not for the NIP problem. To minimize the system response delay, Sun et al. [31] proposed an improved genetic algorithm that avoids infeasible solutions by using integer coding. Although the algorithm in [31] can solve the NIP problem, it still faces the disadvantage of long solution times for problems with large solution spaces. In addition, the above algorithms [29–31] cannot meet the long-term performance requirements in a stochastic environment.

For this, some researchers applied Lyapunov technique to ensure long-term performance. Kuo et al. [32] explored the problem of energy consumption while satisfying the QoS requirement. The multi-stage problem is transformed into a series of single time-slot subproblems by Lyapunov optimization. Then, a gain-based heuristic algorithm is proposed to obtain solutions to the subproblems. To optimize the average response time within an infinite time slot range, Tang et al. [33] proposed two greedy heuristic methods in a drift-plus-penalty-based algorithm to find approximate optimal solutions. Combining Lyapunov optimization techniques with heuristic algorithms can obtain an approximate solution in a short period of time while ensuring the stability of the system. Heuristic algorithms need to be designed based on the problem itself, so the algorithms mentioned in [32,33] cannot directly solve the formulated problem in this article.

Different from previous studies, in terms of task migration, we determine the optimal number of migration tasks to ensure load balancing among RSUs; regarding result feedback, since vehicle locations after task completion remain unknown before offloading decision generation, we design a vehicle trajectory prediction module to obtain future vehicle trajectory information; in algorithm design, we integrate the Lyapunov framework with the heuristic algorithm to efficiently solve the formulated large-scale NIP. The comparisons between our study and existing studies are summarized in Table 1.

3. System model and problem formulation

In this paper, we consider a system model for a MEC-assisted vehicular network as shown in Fig. 1.

In this system model, there are M vehicles and N RSUs, where the vehicles and RSUs sets are denoted as $\mathcal{M} = \{1, 2, \ldots, M\}$ and $\mathcal{N} = \{1, 2, \ldots, N\}$, respectively. The tasks generated by vehicles can be processed locally or transmitted to the RSU layer for processing via the V2I link. To ensure load balancing of RSUs, RSUs with high-load

Table 1
Comparison of our and existing studies.

Ref.	Task migration	Load balancing	Result feedback	Cross-regional mobility	Trajectory prediction	Problem type	Algorithm
[22]						NLP	Mathematical
[10]			√		Kinesiology	MINLP	DRL
[11]				√		MINLP	DRL
[12]	√			√		NLP	Mathematical
[13]	√			√		MILP	DRL
[14]	√			√	Kinesiology	NLP	DRL
[15]		√				ILP	Hungarian matching
[16]		√				ILP	Hungarian matching
[19]			√			NIP	Learning
[20]			√			NIP	Learning
[21]			√	√	Probability model	MINLP	Mathematical
[18]			√	√	Probability model	MINLP	Game
[23]						MINLP	DRL
[24]	√					MINLP	DRL
[25]						NLP	MADRL
[26]						NLP	MADRL
[27]					Probability model	NLP	MADRL
[28]	√		√			MINLP	DRL
[29]						ILP	Matching-based
[30]						ILP	Matching-based
[31]					Probability model	NIP	Heuristic
[32]						MINLP	Lyapunov+Heuristic
[33]						NIP	Lyapunov+Heuristic
Our study	√	√	√	√	Data-driven	NIP	Lyapunov+Heuristic

NLP: Nonlinear programming; MINLP: Mixed integer nonlinear programming; MILP: Mixed integer linear programming.

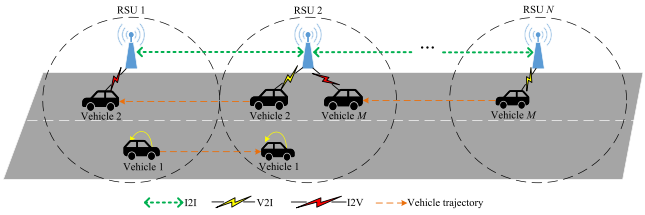


Fig. 1. System model.

will migrate some tasks to RSUs with low-load via I2I link. The results are transmitted from RSU layer to vehicles via the I2V link.

We adopt a time-slotted model where the total optimization period is divided into T time slots. Referring to works [34,35], we assume that each vehicle generates only one task within each time slot, and the generated task is processed within the time slot (including task upload, migration, computation, and result feedback). Based on above assumption, the system will not have any leftover tasks from the previous time slot in the next time slot, i.e., there is no queue backlog in the system. Consider the binary offloading model, a task generated by a vehicle as a whole is either processed locally or offloaded to RSU layer. If there are significant load differences between RSUs, the system will perform task migration via I2I link. During the migration process, the system determines the number of emigration or immigration tasks for each RSU based on their load level to ensure load balancing. Then, the system determines which tasks should be migrated to which RSU to minimize total computation cost. Let matrix $\boldsymbol{O}(t)$, $0 \le t \le T$, denote the task offloading decisions for all vehicles in time slot t. $O(t)$ is given as follows:

$$\boldsymbol{O}(t) = \begin{bmatrix} o_{1,0}(t) & o_{1,1}(t) & \cdots & o_{1,N}(t) \\ o_{2,0}(t) & o_{2,1}(t) & \cdots & o_{2,N}(t) \\ \cdots & \cdots & \cdots & \cdots \\ o_{M,0}(t) & o_{M,1}(t) & \cdots & o_{M,N}(t) \end{bmatrix}_{M\times(N+1)}. \tag{1}$$

Matrix $\boldsymbol{O}(t)$ has M rows, each taking $N+1$ elements. Each element takes a value of 0 or 1, representing the offloading selection of each vehicle. If a task generated by vehicle m ($m \in \mathcal{M}$) is executed locally, we have $o_{m,0}(t) = 1$, otherwise $o_{m,0}(t) = 0$. If the task generated by vehicle m is finally processed by RSU n after uploading and migration, we have $o_{m,n}(t) = 1$ ($n \in \mathcal{N}$); otherwise $o_{m,n}(t) = 0$. Due to the indivisibility of tasks, the tasks generated by each vehicle can only be executed in one place, which means that the sum of any row of matrix $\boldsymbol{O}(t)$ is 1.

We denote D_{in} (in bits) as size of task input data, C (in CPU cycles per bit) as the processing density, and $D_{\text{out}} = \delta D_{\text{in}}$ as size of task result data, where δ is the result/input data size ratio.

3.1. Local processing model

Given the available processing frequency $f_m^{\text{lo}}(t)$ for the CPU of vehicle m in time slot t, the local computation delay $T_m^{\text{loc}}(t)$ of a task can be calculated as follows:

$$T_m^{\text{loc}}(t) = \frac{D_{\text{in}} C}{f_m^{\text{lo}}(t)}, \tag{2}$$

the energy consumption $E_m^{\text{lo}}(t)$ of a vehicle to process a task in time slot t can be calculated as

$$E_m^{\text{lo}}(t) = T_m^{\text{loc}}(t) P_m, \tag{3}$$

where P_m is the computation power for the CPU of vehicle m.

3.2. RSU processing model

We assume that the orthogonal frequency division multiple access (OFDMA) technology is applied to the V2I communication link. The effective bandwidth of each subchannel is B_{V2I}. Since subchannels are orthogonal to each other, there is no interference between the subchannels under the same RSU. However, interference caused by channel reuse still persists. Common network deployment schemes typically allocate different frequency bands to adjacent RSUs. Channel reuse may occur in non-adjacent RSUs, but the inter-RSU interference is small due to the relatively large distances between these RSUs. Therefore, the impact of inter-RSU interference on system performance is quite limited. We use $I_m^{\text{V2I}}(t)$ to represent the average V2I link interference caused when vehicles under other RSUs use the same communication frequency as vehicle m. Let $a_m(t)$, $a_m(t) \in \mathcal{N}$, be the NO. of the RSU contacting with vehicle m. According to Shannon theory, the V2I link transmission rate of vehicle m in time slot t is given as follows:

$$R_m^{\text{V2I}}(t) = B_{\text{V2I}} \log_2 \left(1 + \frac{P_m^{\text{V2I}} h_m(t)}{\sigma^2 + I_m^{\text{V2I}}}\right), \tag{4}$$

where P_m^{V2I} denotes the transmission power of vehicle m, $h_m(t)$ denotes the channel gain between vehicle m and RSU $a_m(t)$, σ^2 denotes the power of Gaussian white noise.

Then, we obtain upload delay $T_m^{\mathrm{V2I}}(t)$ and upload energy consumption $E_m^{\mathrm{V2I}}(t)$ for vehicle m in time slot t as follows:

$$T_m^{\mathrm{V2I}}(t) = \frac{D_{\mathrm{in}}}{R_m^{\mathrm{V2I}}(t)}, \tag{5}$$

$$E_m^{\mathrm{V2I}}(t) = P_m T_m^{\mathrm{V2I}}(t). \tag{6}$$

RSU n hosts k_n homogeneous virtual machines, each with an average computation frequency f_n^{RSU}. Assuming that tasks arrive at a RSU following a Poisson process and the processing time of a task on a virtual machine follows an exponential distribution. We build $M/M/k_n$ queue to investigate computation delay of tasks at RSU n.

We denote Λ as the length of one time slot. The arrival rate of tasks at RSU n in time slot t can be given by

$$\lambda_n(t) = \frac{\sum_{i\in\mathcal{M}} o_{i,n}(t)}{\Lambda}, \tag{7}$$

where $\sum_{i\in\mathcal{M}} o_{i,n}(t)$ denotes the number of tasks ultimately processed at RSU n after V2I link transmission and I2I link migration.

We denote $T_n^{\mathrm{que}}(t)$ as the computation delay of a task at RSU n in time slot t. Applying the steady-state result of $M/M/k_n$ queue [36], $T_n^{\mathrm{que}}(t)$ can be given by

$$T_n^{\mathrm{que}}(t) = \frac{\rho_n(t)\left(k_n\rho_n(t)\right)^{k_n}\left(\lambda_n(1-\rho_n(t))^2 k_n!\right)^{-1}}{\sum_{i=0}^{k_n-1}\frac{(k_n\rho_n(t))^i}{i!} + \frac{(k_n\rho_n(t))^{k_n}}{k_n!(1-\rho_n(t))}}, \tag{8}$$

where $\rho_n(t) = \lambda_n(t)/(k_n\mu_n)$ denotes the load level of RSU n in time slot t and μ_n denotes the computation rate of a virtual machine at RSU n. To ensure the stability of the task computation queue at RSU n, a constraint is expressed as

$$\rho_n(t) < 1, \ \forall n, \ t. \tag{9}$$

We denote $T_n^{\mathrm{com}}(t)$ as the average computation delay of a task at RSU n in time slot t, which can be given by

$$T_n^{\mathrm{com}}(t) = \frac{1}{\mu_n} = \frac{D_{\mathrm{in}}C}{f_n^{\mathrm{RSU}}}. \tag{10}$$

Combining Eqs. (8) and (10), the service delay $T_n^{\mathrm{ser}}(t)$ of a task at RSU n in time slot t can be given by

$$T_n^{\mathrm{ser}}(t) = T_n^{\mathrm{que}}(t) + T_n^{\mathrm{com}}(t). \tag{11}$$

We use standard deviation $\Phi(t)$ to characterize the load balancing level of all RSUs in time slot t, $\Phi(t)$ is given by

$$\Phi(t) = \sqrt{\frac{1}{N}\sum_{n\in\mathcal{N}}(\rho_n(t) - \bar{\rho}(t))^2}, \tag{12}$$

where $\Phi(t)$ decrease indicates an improvement in load balancing level. $\bar{\rho}(t)$ is the average load level of total system. $\bar{\rho}(t)$ is given by

$$\bar{\rho}(t) = \frac{\sum_{n\in\mathcal{N}}\rho_n(t)}{N}. \tag{13}$$

Let P_n^{idle} be the power consumption of an idle virtual machine in RSU n, P_n^{max} be the maximum power consumption of a full load virtual machine in RSU n. According to [37], the average power consumption $P_n^{\mathrm{ave}}(t)$ of RSU n in time slot t is given by

$$P_n^{\mathrm{ave}}(t) = k_n\left(P_n^{\mathrm{idle}} + \rho_n^2(t)(P_n^{\mathrm{max}} - P_n^{\mathrm{idle}})\right). \tag{14}$$

The total computation energy consumption $E_n(t)$ of RSU n in time slot t can be calculated as

$$E_n(t) = P_n^{\mathrm{ave}}(t)\Delta. \tag{15}$$

Let $\hat{a}_m(t)$ be the NO. of the RSU ultimately processing the task generated by vehicle m in time slot t. The number $\xi^{\mathrm{in}}(t)$ of hops from RSU $a_m(t)$ to RSU $\hat{a}_m(t)$ is given by

$$\xi^{\mathrm{in}}(t) = \left\lceil \frac{l(a_m(t), \hat{a}_m(t))}{L_{\mathrm{RSU}}} \right\rceil, \tag{16}$$

where $l(a_m(t), \hat{a}_m(t))$ denotes the distance between RSU $a_m(t)$ and RSU $\hat{a}_m(t)$, L_{RSU} denotes the distance between two adjacent RSUs, $\lceil\cdot\rceil$ denotes rounding up operation.

Given the transmission rate R_{I2I} of the I2I link, migration delay $T_m^{\mathrm{mig,in}}(t)$ of a task generated by vehicle m in time slot t can be calculated as

$$T_m^{\mathrm{mig,in}}(t) = \xi^{\mathrm{in}}(t)\frac{D_{\mathrm{in}}}{R_{\mathrm{I2I}}}. \tag{17}$$

Given the transmission power consumption P^{tr} of the I2I link, migration energy consumption $E_m^{\mathrm{mig}}(t)$ of a task generated by vehicle m in time slot t can be given by

$$E_m^{\mathrm{mig}}(t) = P^{\mathrm{tr}} T_m^{\mathrm{mig,in}}(t). \tag{18}$$

Let $\bar{a}_m(t)$ be the NO. of the RSU closest to vehicle m after a task is completed. $\bar{a}_m(t)$ can be obtained by the vehicle trajectory prediction module, which will be presented in Section 4. The number $\xi^{\mathrm{out}}(t)$ of hops from RSU $\hat{a}_m(t)$ to RSU $\bar{a}_m(t)$ is given by

$$\xi^{\mathrm{out}}(t) = \left\lceil \frac{l(\hat{a}_m(t), \bar{a}_m(t))}{L_{\mathrm{RSU}}} \right\rceil, \tag{19}$$

where $l(\hat{a}_m(t), \bar{a}_m(t))$ denotes the distance between RSU $\hat{a}_m(t)$ and RSU $\bar{a}_m(t)$.

Denote $T_m^{\mathrm{mig,out}}(t)$ as the delay that the computation result transmitted to the nearest RSU of vehicle m in time slot t. $T_m^{\mathrm{mig,out}}(t)$ is given by

$$T_m^{\mathrm{mig,out}}(t) = \xi^{\mathrm{out}}(t)\frac{D_{\mathrm{out}}}{R_{\mathrm{I2I}}}. \tag{20}$$

Similar to the V2I communication link model, we assume that the I2V communication link also applies the OFDMA technology. We use $I_m^{\mathrm{I2V}}(t)$ to represent the average I2V link interference caused when the vehicles under other RSUs use the same communication frequency as vehicle m. Given the bandwidth B^{I2V} of the I2V link, the channel gain $h_{\bar{a}_m}(t)$ between RSU $\bar{a}_m(t)$ and vehicle m, and the power consumption P^{I2V} of the I2V link, the I2V link of vehicle m in time slot t can be calculated as

$$R_m^{\mathrm{I2V}}(t) = B_{\mathrm{I2V}}\log_2\left(1 + \frac{P^{\mathrm{I2V}}h_{\bar{a}_m}(t)}{\sigma^2 + I_m^{\mathrm{I2V}}}\right). \tag{21}$$

The delay $T_m^{\mathrm{mig,v}}(t)$ that the computation result transmitted from RSU $\bar{a}_m(t)$ to vehicle m in time slot t can be given by

$$T_m^{\mathrm{mig,v}}(t) = \frac{D_{\mathrm{out}}}{R_m^{\mathrm{I2V}}(t)}. \tag{22}$$

Combining Eqs. (20) and (22), the result feedback delay $T_m^{\mathrm{back}}(t)$ of a task in time slot t can be given by

$$T_m^{\mathrm{back}}(t) = T_m^{\mathrm{mig,out}}(t) + T_m^{\mathrm{mig,v}}(t), \tag{23}$$

then energy consumption $E_m^{\mathrm{back}}(t)$ of result feedback for a task generated by vehicle m in time slot t can be given by

$$E_m^{\mathrm{back}}(t) = P^{\mathrm{tr}} T_m^{\mathrm{mig,out}}(t) + P^{\mathrm{I2V}} T_m^{\mathrm{mig,v}}(t). \tag{24}$$

Combining Eqs. (2), (5), (11), (17), and (23), the task delay in time slot t can be given by

$$T_m(t) = o_{m,0}(t)T_m^{\mathrm{loc}}(t) + \left(1 - o_{m,0}(t)\right)\left(T_m^{\mathrm{V2I}}(t) + T_{\hat{a}_m(t)}^{\mathrm{ser}}(t) + T_m^{\mathrm{mig,in}}(t) + T_m^{\mathrm{back}}(t)\right), \tag{25}$$

then the total delay $T(t)$ of the system in time slot t can be given by

$$T(t) = \sum_{m\in\mathcal{M}} T_m(t). \tag{26}$$

Combining Eqs. (3) and (6), the total energy consumption $E^{\mathrm{V}}(t)$ of all vehicles in time slot t is calculated by

$$E^{\mathrm{V}}(t) = \sum_{m\in\mathcal{M}} o_{m,0}(t)E_m^{\mathrm{lo}}(t) + (1 - o_{m,0}(t))E_m^{\mathrm{V2I}}(t). \tag{27}$$

Combining Eqs. (15), (18), and (24), the total energy consumption $E^{\mathrm{R}}(t)$ of all RSUs in time slot t is calculated by

$$E^{\mathrm{R}}(t) = \sum_{m\in\mathcal{M}} \left((1 - o_{m,0}(t)) E_m^{\mathrm{mig}}(t) + E_m^{\mathrm{back}}(t) \right) + \sum_{n\in\mathcal{N}} E_n(t). \tag{28}$$

Considering that vehicles need to be powered by batteries, while RSUs have a stable power source due to being connected to the power grid, we assign two weight parameters, β^{V} and β^{E}, using to represent the energy consumption cost $E(t)$ in time slot t. $E(t)$ is calculated by

$$E(t) = \beta^{\mathrm{V}} E^{\mathrm{V}}(t) + \beta^{\mathrm{E}} E^{\mathrm{R}}(t), \tag{29}$$

where $\beta^{\mathrm{V}} \in [0,1]$, $\beta^{\mathrm{E}} \in [0,1]$, and $\beta^{\mathrm{V}} + \beta^{\mathrm{E}} = 1$.

The total computation cost $Y(t)$ of vehicular networks system is calculated as the weighted sum of total delay and total energy consumption as follows:

$$Y(t) = \alpha^{\mathrm{T}} T(t) + \alpha^{\mathrm{E}} E(t), \tag{30}$$

where $\alpha^{\mathrm{T}} \in [0,1]$ and $\alpha^{\mathrm{E}} \in [0,1]$ are the trade-off parameters for total delay and total energy consumption, respectively, and $\alpha^{\mathrm{T}} + \alpha^{\mathrm{E}} = 1$.

3.3. Problem formulation

Given the delay of vehicle tasks and the level of load balancing between RSUs, we jointly consider hybrid computation offloading decisions $\boldsymbol{O}(t)$ (i.e., vehicle-to-RSU offloading and RSU-RSU offloading) to minimize long-term computation cost. For this, we formulate optimization problem **P0** as follows:

$$\mathbf{P0}: \min_{\boldsymbol{O}(t),\forall t} \quad \frac{1}{T}\sum_{t=0}^{T-1} Y(t)$$

$$\text{s.t.} \quad \frac{1}{T}\sum_{t=0}^{T-1} \rho_n(t) \le \rho_n^{\mathrm{th}}, \ \forall n \tag{31a}$$

$$\rho_n(t) < 1, \ \forall n, \forall t \tag{31b}$$

$$T_m(t) < T^{\mathrm{th}}, \ \forall m, \forall t \tag{31c}$$

$$\boldsymbol{\Phi}(t) \le \boldsymbol{\Phi}^{\max}, \ \forall t \tag{31d}$$

$$o_{m,n}(t) \in \{0,1\}, \ \forall m, \ \forall n, \ \forall t \tag{31e}$$

$$\sum_{n\in\mathcal{N}} o_{m,n}(t) = 1, \ \forall m, \ \forall n, \ \forall t, \tag{31f}$$

where (31a) is the constraint of long-term load level for each RSU. (31b) ensures that each RSU is not overloaded, i.e., task arrival rate $\lambda_n(t)$ should not exceed computation rate $k_n\mu_n$. (31c) ensures that each task must be completed within the specified time T^{th}. (31d) indicates that the load balancing level of all RSUs is less than $\boldsymbol{\Phi}^{\max}$, where the lower $\boldsymbol{\Phi}(t)$, the more balanced the edge layer. (31e) and (31f) are binary offload constraints that guarantee that a task can only be processed locally or offloaded to a RSU.

4. Problem solution

In this section, we present the framework design of LNOHO algorithm. Firstly, we decouple the original optimization problem into multiple subproblems. Then, we propose a vehicle trajectory prediction module to obtain the NO. of the RSU closest to vehicle m after a task is completed. Finally, we propose a LMH algorithm to solve each subproblem.

4.1. Problem decoupling

With long-term coupling constraint (31a) consisting of decisions at different time slots, solving problem **P0** requires complete information. Predicting future information is difficult due to the randomly changing network environment. For making online decisions, we adopt the Lyapunov optimization framework to transform the original problem into multiple per-slot deterministic subproblems. For each subproblem, the online solution only relies on the information about the current time slot.

We build a virtual queue $q_n(t)$ to process constraint (31a) as follows:

$$q_n(t+1) = \max\left(q_n(t) + \rho_n(t) - \rho_n^{\mathrm{th}}, 0\right), \tag{32}$$

where $q_n(t)$ is the queue length in time slot t and $q_n(0) = 0$. The value of $q_n(t)$ indicates how much the load level of RSU n exceeds the specified value ρ_n^{th} in time slot t. To ensure that constraint (31a) is satisfied, the virtual queue should be stable, i.e., $\lim_{T\to\infty} \frac{q_n(T)}{T} = 0$.

We define a Lyapunov function as follows:

$$L(\boldsymbol{\Theta}(t)) = \frac{1}{2}\sum_{n\in\mathcal{N}} q_n^2(t), \tag{33}$$

where $\boldsymbol{\Theta}(t) = \{q_n(t)\}_{n\in\mathcal{N}}$.

The conditional Lyapunov drift function $\Delta(\boldsymbol{\Theta}(t))$ is then given by

$$\Delta(\boldsymbol{\Theta}(t)) = \mathbb{E}[L(\boldsymbol{\Theta}(t+1)) - L(\boldsymbol{\Theta}(t))|\boldsymbol{\Theta}(t)]. \tag{34}$$

By summing the total computation cost (as a penalty component) into (34) to obtain the drift plus penalty function as follows:

$$\Delta(\boldsymbol{\Theta}(t)) + V\mathbb{E}[Y(t)|\boldsymbol{\Theta}(t)],$$

where V is a positive constant used to control the trade-off between total computation cost and RSUs utilization. The core idea of the Lyapunov optimization framework is to minimize the upper bound of the drift plus penalty function. Next, we will derive the upper bound of the drift plus penalty function.

$$\begin{aligned}
&\Delta(\boldsymbol{\Theta}(t)) + V\mathbb{E}[Y(t)|\boldsymbol{\Theta}(t)] \\
&= \mathbb{E}[L(\boldsymbol{\Theta}(t+1)) - L(\boldsymbol{\Theta}(t))|\boldsymbol{\Theta}(t)] + V\mathbb{E}[Y(t)|\boldsymbol{\Theta}(t)] \\
&= \mathbb{E}\left[\frac{1}{2}\sum_{n\in\mathcal{N}} q_n^2(t+1) - \frac{1}{2}\sum_{n\in\mathcal{N}} q_n^2(t)|\boldsymbol{\Theta}(t)\right] \\
&\quad + V\mathbb{E}[Y(t)|\boldsymbol{\Theta}(t)] \\
&= \mathbb{E}\left[\frac{1}{2}\sum_{n\in\mathcal{N}} \max\left(\left(q_n(t) + \rho_n(t) - \rho_n^{\mathrm{th}}, 0\right)\right)^2\right. \\
&\quad \left. - \frac{1}{2}\sum_{n\in\mathcal{N}} q_n^2(t)|\boldsymbol{\Theta}(t)\right] + V\mathbb{E}[Y(t)|\boldsymbol{\Theta}(t)] \\
&\le \mathbb{E}\left[\frac{1}{2}\sum_{n\in\mathcal{N}} \left(q_n(t) + \rho_n(t) - \rho_n^{\mathrm{th}}\right)^2 - \frac{1}{2}\sum_{n\in\mathcal{N}} q_n^2(t)|\boldsymbol{\Theta}(t)\right] \\
&\quad + V\mathbb{E}[Y(t)|\boldsymbol{\Theta}(t)] \\
&= \frac{1}{2}\sum_{n\in\mathcal{N}} \left(\rho_n(t) - \rho_n^{\mathrm{th}}\right)^2 + \mathbb{E}\left[\sum_{n\in\mathcal{N}} q_n(t)\left(\rho_n(t) - \rho_n^{\mathrm{th}}\right)\right. \\
&\quad \left. + VY(t)|\boldsymbol{\Theta}(t)\right] \\
&\le \Omega + \mathbb{E}\left[\sum_{n\in\mathcal{N}} q_n(t)\rho_n(t) + VY(t)|\boldsymbol{\Theta}(t)\right],
\end{aligned}$$

where $\Omega = \frac{1}{2}\sum_{n\in\mathcal{N}} (\rho_n^{\max} - \rho_n^{\mathrm{th}})^2$ is a constant, $\rho_n^{\max}$ is the maximum value of $\rho_n(t)$. In each time slot, we apply the technique of opportunistically minimizing an expectation to transform the continuous time slot optimization problem **P0** into the deterministic per-slot subproblem **P1** as follows:

$$\mathbf{P1}: \min_{\boldsymbol{O}(t)} \quad \sum_{n\in\mathcal{N}} q_n(t)\rho_n(t) + VY(t)$$

$$\text{s.t.} \quad \text{(31b)} - \text{(31f)}. \tag{35}$$

Based on the above analysis, we propose a LNOHO algorithm as shown in Algorithm 1.

Algorithm 1 LNOHO Algorithm

Input: Parameter V, Virtual queue $q_n(0) = 0,\ \forall n \in \mathcal{N}$.
Output: Offloading decisions $\{\boldsymbol{O}(t)\}_{t\in\mathcal{T}}$.
1: **for** $t = 0, 1, ..., T-1$ **do**
2: Solve problem **P1**
3: Update $q_n(t)$ based on (32), $\forall n \in \mathcal{N}$
4: **end for**

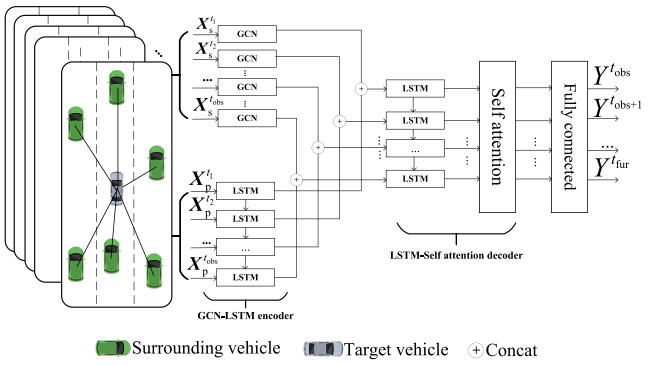


Fig. 2. Architecture of encoder–decoder prediction module.

4.2. Vehicle trajectory prediction module

In complex and changing traffic environments, vehicle trajectories are not only related to their own motion states but also to the motion states of surrounding vehicles. For example, if a vehicle passes by on the left side of a target vehicle, the target vehicle will not drive to the left. To this end, we propose a data-driven sequence-to-sequence based deep neural network to predict the future trajectory of vehicles. The prediction module adopts an encoder–decoder architecture, as shown in Fig. 2.

The encoder consists of GCN networks and LSTM networks. The GCN networks are used to extract historical interaction information between the target vehicle and surrounding vehicles. We select the target vehicle as the center, with 50 meters in front and behind, and one lane on the left and right as the interaction area. Within this area, select the vehicles in front, behind, left front, left behind, right front, and right behind of the target vehicle as its surrounding vehicles. The input of the GCN network is the historical location information of the target vehicle and its surrounding vehicles under $t \in [t_1, t_{\text{obs}}]$, i.e. $\{X_s^{t_1}, X_s^{t_2}, \ldots, X_s^{t_{\text{obs}}}\}$. t_1 and t_{obs} are the starting and ending time instants of historical information, respectively. LSTM can mine the correlation between input samples, which is usually used to process sequence data. For encoders, the LSTM network is used to extract the historical kinematic features of the target vehicle at $t \in [t_1, t_{\text{obs}}]$ with the input $\{X_p^{t_1}, X_p^{t_2}, \ldots, X_p^{t_{\text{obs}}}\}$.

The decoder is designed based on LSTM and self attention networks. LSTM networks of the decoder perform sequential processing for the temporal feature sequence output by the encoder and then send processing results to self attention networks. Self attention networks focus on the degree of correlation between the elements of each time step in the input sequence and the elements of other time instants, thereby highlighting the information of important elements. Finally, the trajectory $\{Y_s^{t_{\text{obs}}}, Y_s^{t_{\text{obs}+1}}, \ldots, Y_s^{t_{\text{fur}}}\}$ of the target vehicle at $t \in [t_{\text{obs}}, t_{\text{fur}}]$ is obtained through a fully connected network. t_{obs} and t_{fur} are the starting and ending time points of trajectory prediction information, respectively.

To validate the effectiveness of the proposed vehicle trajectory prediction module in an urban traffic scenario, while ensuring consistency with the traffic scenario used in subsequent comparison experiments, we select simulation data from Luxembourg SUMO traffic scenario [38]

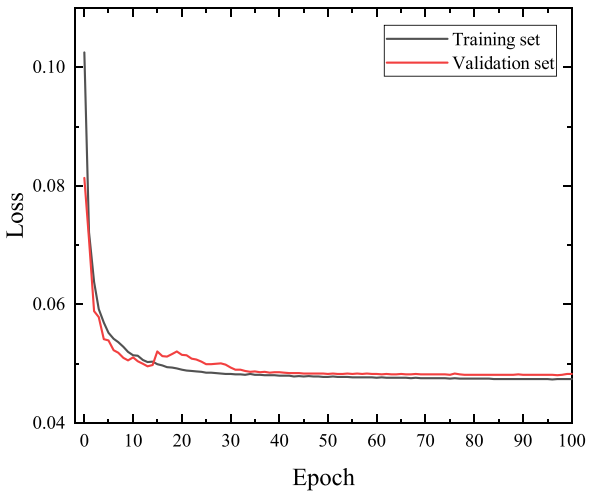


Fig. 3. The loss value of the proposed prediction module on the training and the validation set.

based on real-world road networks as the dataset. This scenario features a common urban topological structure as well as realistic traffic demands and mobility patterns. More importantly, the movement trajectories and speed distributions generated by the simulation data closely resemble those observed in real floating car collected in Luxembourg. All the computations are in Pycharm 2021 with Python 3.9.7 platform on a workstation equipped AMD Ryzen 9 5900X 3.7 GHz CPU and 32 GB of memory. The mobility of vehicles is simulated through the simulation of urban mobility (SUMO). As a transportation simulation platform, SUMO provides users a convenient development interface called Traci. Through this interface, we can easily obtain real-time information on all vehicles and use it as a dataset to verify the effectiveness of the prediction module.

We select the trajectory data of vehicles passing between 18:00 and 19:00, with a sampling frequency of 5 Hz. The collected information includes 14 data items such as vehicle label, coordinates, speed, acceleration, and travel direction. We select the trajectory of a target vehicle and its surrounding vehicles in the first 10 s to predict the trajectory of the target vehicle in the next 5 s. We select 70% of the data in the dataset as the training set and remaining 30% of the data in the dataset as the validation set. In the encoder, the output of GCN and the hidden layer of LSTM are set to 16. In the decoder, the hidden layer of the LSTM is set to 128, and the weight matrix dimension of the self attention layer is set to 128 * 128. We choose the root-mean-square-error (RMSE) of the true trajectory value and the predicted value of the target vehicle as the loss function. The module is trained using the Adam optimizer. The learning rate is 0.01. The loss values during the training and prediction processes are shown in Fig. 3.

From Fig. 3, we see that after training for 100 epochs, the prediction module performs well on both the training and validation sets.

We compare the performance of our proposed prediction module with the following 3 prediction modules:

- D-LSTM [39]: a dual LSTM module is designed to realize vehicle trajectory prediction from two dimensions of time and space.
- GCN-LSTM [40]: a LSTM module integrated with GCN is designed to model the spatio-temporal interaction between a target vehicle and its surrounding vehicles.
- SAT-LSTM [41]: a LSTM module integrated with self attention network is designed to model spatial and temporal dependencies among vehicles, respectively.

The RMSE of result feedback for different modules under prediction horizon from 1 to 5 s is shown in Table 2. We can find that our proposed

Table 2
RMSE for different modules under different prediction horizon.

Module	RMSE				
	1 s	2 s	3 s	4 s	5 s
Proposed	0.0233	0.0305	0.0421	0.0544	0.0712
D-LSTM	0.0271	0.0364	0.0509	0.0687	0.0918
GCN-LSTM	0.0253	0.0323	0.0471	0.0630	0.0833
SAT-LSTM	0.0243	0.0329	0.0458	0.0591	0.0805

Table 3
Ablation study.

Ablation	RMSE				
	1 s	2 s	3 s	4 s	5 s
Proposed	0.0233	0.0305	0.0421	0.0544	0.0712
-GCN	0.0281	0.0366	0.0511	0.0695	0.0925
-LSTM	0.0246	0.0326	0.0453	0.0591	0.0783
-Attention	0.0265	0.0353	0.0489	0.0668	0.0903

vehicle trajectory prediction module achieves the lowest RMSE under prediction horizon from 1 to 5 s. Moreover, based on the data in the Table 2, we can calculate that the RMSE of proposed module is lower than the other modules by an average of 8.68%, 9.69%, 11.99%, 14.13%, 16.17% for prediction horizon from 1s to 5s. This shows that the RMSE of the three modules increases significantly as the prediction horizon increases. In contrast, our proposed module increases slower, which performs well with long prediction horizons. Our proposed vehicle trajectory prediction module can more accurately evaluate the delay and energy consumption in the result feedback process .

To evaluate the impact of each component in the proposed vehicle trajectory prediction module on prediction performance, we conduct the ablation study. The detailed operation instructions are as follows:

- -GCN: Remove GCN module in encoder from our model.
- -LSTM: Remove LSTM module in encoder from our model.
- -Attention: Remove self-attention module in decoder from our model.

As shown in Table 3, the GCN module in the encoder, which processes vehicle interaction information, has the greatest impact on prediction performance, while the LSTM module in the decoder has the least influence. This indicates that interaction information between vehicles plays a non-negligible role in trajectory prediction. Moreover, the parallel processing enabled by the self-attention mechanism also contributes to more stable prediction performance.

4.3. LMH algorithm

NIP problem **P1** can be mapped to a multidimensional 0–1 knapsack problem. Due to the NP-hard characteristic [42,43], making optimal offloading decisions is a challenging issue. Moreover, the size of the **P1** solution space is N^M. When N and M increase, the size of the solution space increases exponentially. To achieve efficient solution, we propose the LMH algorithm to solve per-slot subproblem **P1** follows three steps.

(1) Determine whether a task is processed locally or in the RSU: We note that if a task is processed locally, it will not cause load on RSUs (i.e. virtual queues are smaller) and will not generate additional delay and energy consumption except for computing the task. Therefore, local processing is a priority. Considering constraint (31c), we have

$$\begin{cases} o_{m,0}(t)=1, & \text{if } T_m^{\text{lo}}(t)\le T^{\text{th}} \\ o_{m,a_m}(t)=1, & \text{otherwise} \end{cases}, \ \forall m\in\mathcal{M}. \tag{36}$$

According to (36), we can get the number $\kappa_n(t)=\sum_{m\in\mathcal{M}} o_{m,n}(t)$ of vehicles uploading task to RSU n.

(2) Determine the number of emigration tasks or immigration tasks for RSUs: For high-load and low-load RSUs, we use set $\Gamma=\{|\gamma_1|,|\gamma_2|,\dots,|\gamma_N|\}$ to represent the number of emigration tasks ($\gamma_n<0, n\in\mathcal{N}$) or immigration tasks ($\gamma_n>0, n\in\mathcal{N}$), where $|\cdot|$ represents the absolute value. Load level $\rho_n(t)$ is also given by

$$\rho_n(t)=\frac{\kappa_n(t)+\gamma_n}{\Lambda k_n\mu_n}. \tag{37}$$

In order to determine elements in set Γ, in each iteration, we denote $\tau_{\max}$ and $\tau_{\min}$ as the NO. of the RSUs with the highest and lowest load levels, respectively, where $\tau_{\max},\tau_{\min}\in\mathcal{N}$. Then, RSU $\tau_{\max}$ will migrate tasks to RSU $\tau_{\min}$, and the number ϵ of tasks that need to be migrated can be found by the following system of equations:

$$\begin{cases} \dfrac{\rho_{\tau_{\max}}+\rho_{\tau_{\min}}}{2}=\dfrac{\kappa_{\tau_{\max}}(t)+\gamma_{\tau_{\max}}-\epsilon_{\tau_{\max}}}{\Delta k_{\tau_{\max}}\mu_{\tau_{\max}}} \\ \dfrac{\rho_{\tau_{\max}}+\rho_{\tau_{\min}}}{2}=\dfrac{\kappa_{\tau_{\min}}(t)+\gamma_{\tau_{\min}}+\epsilon_{\tau_{\min}}}{\Delta k_{\tau_{\min}}\mu_{\tau_{\min}}} \\ \epsilon=\min(\epsilon_{\tau_{\max}},\epsilon_{\tau_{\min}}) \end{cases}, \tag{38}$$

where, $\epsilon_{\max}\ge 0, \epsilon_{\min}\ge 0$.

(38) can reduce the load difference between RSU $\tau_{\max}$ and RSU $\tau_{\min}$ to the greatest extent possible, thereby improving the load balancing level $\Phi(t)$ of all RSUs to the greatest extent possible. After determining the number ϵ of migration tasks, the elements $\gamma_{\tau_{\max}}$ and $\gamma_{\tau_{\min}}$ in set Γ can be iteratively updated by

$$\gamma_{\tau_{\max}}^{[j+1]}=\gamma_{\tau_{\max}}^{[j]}-\epsilon, \tag{39}$$

$$\gamma_{\tau_{\min}}^{[j+1]}=\gamma_{\tau_{\min}}^{[j]}+\epsilon. \tag{40}$$

In addition $\rho_{\tau_{\max}}(t)$ and $\rho_{\tau_{\min}}(t)$ also need to be updated based on (37).

(3) Determine decision matrix: By set Γ, we construct two sets $\mathcal{N}_{\text{em}}=\{n|\gamma_n\le 0\}$ and $\mathcal{N}_{\text{im}}=\{n|\gamma_n>0\}$ to represent the NO. of high-load RSUs and low-load RSUs, respectively. We use set $\Gamma_{\text{em}}=\{|\gamma_n|\ |n\in\mathcal{N}_{\text{em}}\}$ to represent the number of emigration tasks for RSUs in set $\mathcal{N}_{\text{em}}$, and set $\Gamma_{\text{im}}=\{|\gamma_n|\ |n\in\mathcal{N}_{\text{im}}\}$ to represent the number of immigration tasks that need to be migrated for RSUs in set $\mathcal{N}_{\text{im}}$. Then, we need to find out for the tasks uploaded to RSUs belonging to set $\mathcal{N}_{\text{em}}$, which tasks migrating to which the RSU belonging to set $\mathcal{N}_{\text{im}}$ will result in the lowest total computation cost, which will determine the decision matrix. In the step 1, we can obtain the task upload delay $T_m^{\text{V2I}}(t)$, task upload energy consumption $E_m^{\text{V2I}}(t)$, local computation delay $T_m^{\text{lo}}(t)$, and local energy consumption $E_m^{\text{lo}}(t)$ for vehicle m. In the step 2, we can obtain the service delay $T_n^{\text{ser}}(t)$, computation energy consumption $E_n(t)$, and load level $\rho_n(t)$ at RSU n. At this point, decision matrix can be obtained by minimizing the weighted sum of migration cost and feedback cost, i.e. $\alpha(T_m^{\text{mig,in}}(t)+T_m^{\text{back}}(t))+\beta(E_m^{\text{mig,in}}(t)+E_m^{\text{back}}(t))$. If constraint (31d) is satisfied during the iteration process, we will obtain a candidate solution and compare it with the current optimal solution. If the candidate solution is better than the optimal solution, update the optimal solution.

The specific details of the LMH algorithm are given by Algorithm 2.

Algorithm 2 LMH Algorithm

Input: Φ^{th}, J.
Output: Offloading decisions $\boldsymbol{O}^*(t)$.
1: Initialize the decision matrix $\boldsymbol{O}(t)$ according to (36)
2: $\boldsymbol{O}^*(t)\leftarrow\boldsymbol{O}(t)$, $\Gamma\leftarrow\{|\gamma_n|\ |n\in\mathcal{N},\gamma_n=0\}$, $F^*(t)\leftarrow+\infty$
3: Compute $\kappa_n(t),\ \forall n\in\mathcal{N}$
4: **for** $j=0,1,\dots,J$ **do**
5: Find out RSU $\tau_{\max}$ and RSU $\tau_{\min}$
6: Update Γ according to (38)–(40)
7: Compute $\rho_n(t),\ \forall n\in\mathcal{N}$, according to (37)
8: Compute $\Phi(t)$ according to (12), (13)
9: Get $\mathcal{N}_{\text{em}}$, $\mathcal{N}_{\text{im}}$, and Γ_{em} by Γ
10: **if** $\Phi(t)\le\Phi^{\max}$ **then**

while $\mathcal{N}_{\text{em}} == \emptyset$ **do**
  Choose an emigration RSU k ($k \in \mathcal{N}_{\text{em}}$) by using the roulette wheel selection based on probability distribution $p(s) = \frac{|\gamma_s|}{\sum_{i \in \Gamma_{\text{em}}} |\gamma_i|}, s \in \mathcal{N}_{\text{em}}$
  Compute the weighted sum of migration cost and feedback cost for each vehicle in set $\mathcal{M}_k$ to migrate to each RSU in set $\mathcal{N}_{\text{im}}$, and find the minimum weighted sum of migration cost and feedback cost corresponding to vehicle m and RSU n
  Update $\boldsymbol{O}(t)$, $o_{m,n}(t) \leftarrow 1$, $\{o_{m,i}(t) \leftarrow 0\}_{i \neq n}$
  Update Γ_{em}, $\gamma_k \leftarrow \gamma_k + 1$
  $\mathcal{M}_k \leftarrow \mathcal{M}_k \setminus \{m\}$
  if $\gamma_k > 0$ **then**
    $\mathcal{N}_{\text{em}} \leftarrow \mathcal{N}_{\text{em}} \setminus \{k\}$
  end if
 end while
 $F_j(t) \leftarrow \sum_{n \in \mathcal{N}} q_n(t)\rho_n + VY(t)$
 if $F_j(t) < F^*(t)$ **then**
  $F^*(t) \leftarrow F_j(t)$
  $\boldsymbol{O}^*(t) \leftarrow \boldsymbol{O}(t)$
 end if
 end if
end for
return $\boldsymbol{O}^*(t)$

4.4. Computation complexity analysis

LNOHO (Algorithm 1) is a subalgorithm of LMH (Algorithm 2). We first analyze the time complexity of LMH. The time complexity of Lines 1–3 is $O(MN)$. The number of outer loop iterations (Line 4) is J. The time complexity of Lines 5–9 is $O(N)$. The number of inner loop iterations (Line 11) is I_{ga}. The time complexity of Line 12 is $O(|\mathcal{N}_{\text{em}}|)$. The time complexity of Line 13 is $O(|\mathcal{M}_k \parallel \mathcal{N}_{\text{im}}|)$. The time complexity of Lines 14–19 is $O(1)$. Thus, the time complexity of Lines 11–20 is $O(I_{\text{ga}}(|\mathcal{N}_{\text{em}}| + |\mathcal{M}_k \parallel \mathcal{N}_{\text{im}}|))$. The time complexity of Line 21 is $O(I_F)$, where I_F is related to M and N. The time complexity of Lines 22–25 is $O(1)$. Thus, the time complexity of Lines 10–26 is $O(I_{\text{ga}}(|\mathcal{N}_{\text{em}}| + |\mathcal{M}_k \parallel \mathcal{N}_{\text{im}}|) + I_F)$. Then, the time complexity of Lines 4–27 is $O(J(N + I_{\text{ga}}(|\mathcal{N}_{\text{em}}| + |\mathcal{M}_k \parallel \mathcal{N}_{\text{im}}|) + I_F))$. Accordingly, the time complexity of LMH algorithm is $O((M+J)N + JI_{\text{ga}}(|\mathcal{N}_{\text{em}}| + |\mathcal{M}_k \parallel \mathcal{N}_{\text{im}}|) + I_F)$. Finally, the time complexity of LNOHO algorithm is $O(T((M+J)N + JI_{\text{ga}}(|\mathcal{N}_{\text{em}}| + |\mathcal{M}_k \parallel \mathcal{N}_{\text{im}}|) + I_F))$. The runtime of algorithms (Table 4) is presented in the Performance evaluation section.

5. Performance evaluations

To verify the effectiveness of the proposed LNOHO algorithm, we conduct simulation experiments based on a real urban traffic scenario, which is consistent with the scenario in trajectory prediction module. The simulation area for the urban traffic scenario is 12 km^2, with 70 RSUs covering the entire area. The driving speed of the vehicle is 30–70 km/h. In the simulation experiment, we investigate the trend of system performance metrics with the model parameters in terms of number of vehicles, number of time slots, size of task input data, and computation frequency of a virtual machine. The influence of the other key parameters on system performance can be intuitively inferred. The specific speculations are as follows. According to (30), a larger α^{T} tends to minimize delay, whereas a smaller value tends to minimize energy consumption. According to (36), a lower threshold T^{th} prioritizes RSU processing to enhance QoS, whereas a higher threshold T^{th} favors local processing to conserve system energy consumption. According to the drift plus penalty function, too large V will increase the load fluctuation of RSUs, while too small V will increase system cost.

The channel gain $h_m(t)$ and $h_{\tilde{a}_m}(t)$ can be calculated by $128.1 + 37.6\log_{10}(d)$ [46], where d_m (km) represents the distance between vehicle and the RSU. The trade-off parameters for total delay and total energy consumption are set to $\alpha = 0.9$ and $\beta = 0.1$, respectively. The trade-off parameters can be adaptively adjusted according to the specific requirements of vehicular applications. Considering that vehicles typically rely on battery or engine power, to ensure vehicle driving range, the energy weighting parameters for vehicles and RSUs are set to $\beta^{\text{V}} = 0.8$ and $\beta^{\text{E}} = 0.2$, respectively. The more simulation parameters are provided in Table 4.

Table 4
Simulation parameters.

Parameter	Value
Result/input data size ratio δ	100 [18]
Processing density C	800 Hz/bit
Available processing frequency f_m	2.5 – 4.5 GHz
Computation power of vehicle m	10 – 15 W
Power σ^2 of Gaussian white noise	−114 dBm [44]
Transmission power P^{V2I} of V2I link	24 dBm [45]
Number k_n of virtual machines for RSU n	12 – 16
Transmission power of the I2I link	33 dBm
Coverage radius of a RSU	300 m
Bandwidth B^{V2I} of the V2I link	10Mhz [19]
I2I link transmission rate	1000 Mbps
Bandwidth B^{I2V} of the I2V link	10 Mhz [19]
Co-channel interference I_m^{V2I} and I_m^{I2U}	−120 dBm
Power consumption P^{I2V} of I2V link	30 dBm [45]
Length of one time slot Λ	3 s
T^{th}	0.5 s
$\boldsymbol{\Phi}^{\max}$	0.2
Parameter V	0.0002

To verify the effectiveness of the LNOHO algorithm, we compare the LNOHO algorithm with the following state-of-the-art task offloading algorithms:

- Lyapunov ant colony (LAC) algorithm [47]. Applying the Lyapunov optimization framework to transform problem **P0** into **P1**, and then applying AC algorithm to solve **P1**. The number of ants is set to 80, the pheromone importance is set to 1.5, the heuristic factor importance is set to 5, and the pheromone evaporation coefficient is set to 0.2.
- Lyapunov quantum behavior particle swarm optimization (LQBPSO) algorithm [48]. Applying the Lyapunov optimization framework to transform problem **P0** into **P1**, and then applying quantum behavior particle swarm algorithm to solve **P1**. QBPSO algorithm combined with the delta potential well theory of quantum mechanics can enhance the global search ability by increasing the randomness of the particle motion position. The number of particles is set to 80, the inertia weight is set to 0.5, and the learning factor is set to 1.3.
- Lyapunov genetic with directed search (LGDS) algorithm [49]. Applying Lyapunov optimization framework to transform problem **P0** into **P1**, and then applying genetic with directed search algorithm to solve **P1**. Genetic with directed search algorithm introduces the directed search operator into the traditional genetic algorithm to obtain a better approximate solution at a faster rate of convergence. The population size is set to 80, the crossover rate is set to 0.8, and the mutation rate is set to 0.02.
- Lyapunov random (LR) algorithm. Applying Lyapunov optimization framework to transform problem **P0** into **P1**, and then randomly generate offloading decisions.

The reasons for choosing the above baseline algorithms are as follows. Given that the problem formulated in this paper involves a long-term constraint (31a), to make online solutions without requiring future information, we introduce the Lyapunov framework in all compared algorithms to transform problem **P0** into **P1**, aligning with our proposed LNOHO algorithm. Since the decoupled problem **P1** is an NIP with a large solution space, a heuristic algorithm is generally

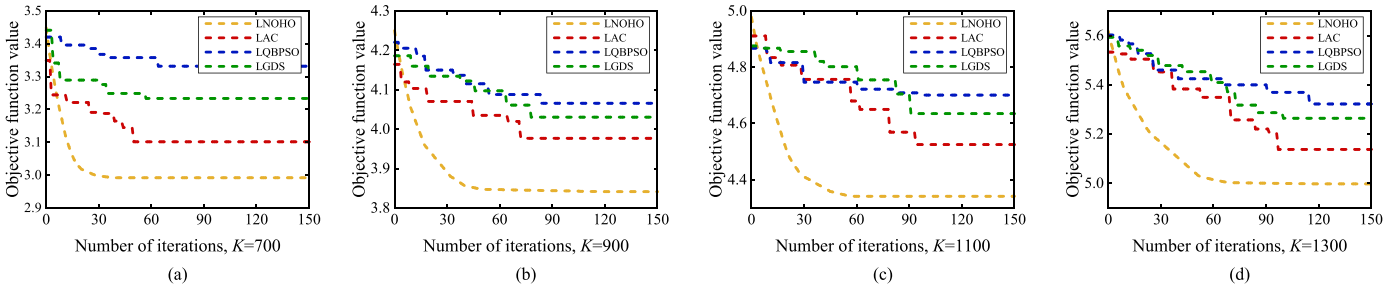


Fig. 4. Convergence performance of LNOHO, LAC, LQBPSO, and LGDS in solving per-slot subproblem **P1**: (a) $K = 700$; (b) $K = 900$; (c) $K = 1100$; (d) $K = 1300$.

Table 5
Comparison of runtime (s) for 150 iterations of solving per-slot subproblem **P1** using different algorithms.

Algorithm	Number of vehicles									
	600	700	800	900	1000	1100	1200	1300	1400	1500
LNOHO	1.23	2.14	2.63	2.74	3.13	3.59	3.86	4.16	5.39	6.39
LAC	253.47	278.12	326.89	358.11	406.74	439.12	490.33	524.58	615.42	659.37
LQBPSO	158.21	179.47	182.19	211.24	225.42	240.21	261.31	292.21	328.31	347.31
LGDS	194.76	226.83	253.67	270.12	305.76	331.98	353.87	379.14	408.65	441.43

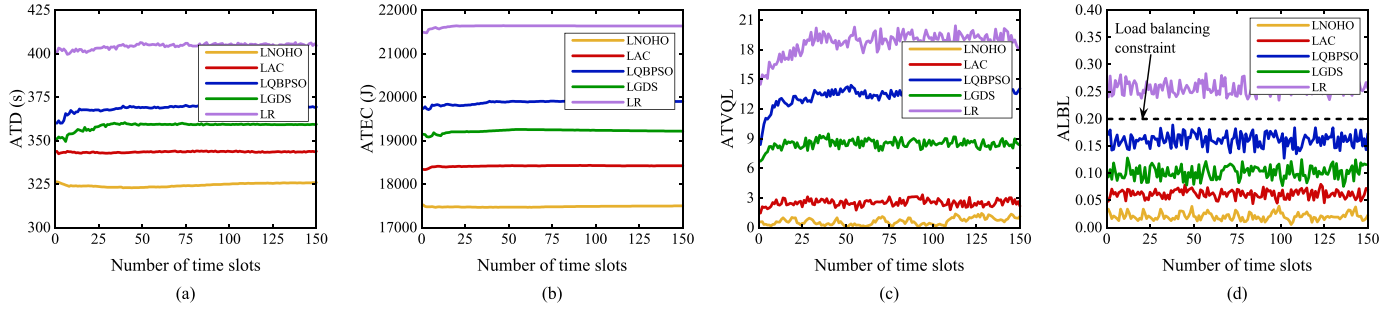


Fig. 5. Trend of different performance indicators over time: (a) ATD; (b) ATEC; (c) ATVQL; (d) ALBL.

a preferable choice. We select ant colony, quantum behavior particle swarm, and genetic with directed search as counterparts. Because these algorithms have strong search capabilities and fast convergence speeds.

We select the following metrics to evaluate the proposed algorithm.

- *Average total delay* (ATD) indicates time average delay of all vehicle tasks, including uploading, computing, migrating, and result feedback. ATD acts as a vital metric to evaluate the response performance of the offloading system. ATD is calculated by

$$\frac{1}{T}\sum_{t=0}^{T-1} T(t). \tag{41}$$

- *Average total energy consumption* (ATEC) indicates time average energy consumption for the system to finish all the vehicle tasks in one time slot. Lower ATEC means that the algorithm achieves higher energy efficiency. ATEC is calculated by

$$\frac{1}{T}\sum_{t=0}^{T-1}\left(E^{\mathrm{V}}(t)+E^{\mathrm{R}}(t)\right). \tag{42}$$

- *Average total virtual queue length* (ATVQL) indicates time average virtual queue lengths for all RSUs. Lower ATVQL indicates smaller load fluctuations and more stable system performance. ATVQL is calculated by

$$\frac{1}{T}\sum_{t=0}^{T-1}\sum_{n\in\mathcal{N}} q_n(t). \tag{43}$$

- *Average load balancing level* (ALBL) indicates time average level of load difference between RSUs. Lower ALBL means a higher level of collaboration between RSUs. ALBL is calculated by

$$\frac{1}{T}\sum_{t=0}^{T-1}\boldsymbol{\Phi}(t). \tag{44}$$

By setting size of task input data $D_{\mathrm{in}} = 2.5$ Mbits and average computation frequency of a virtual machine $f_{\mathrm{RSU}} = 5$ GHz as an example, we compare the convergence performance of LNOHO, LAC, LQBPSO, and LGDS in solving per-slot subproblem **P1** under different number K of vehicles as shown in Fig. 4. From Fig. 4, we can find that four algorithms can achieve convergence within a limited number of iterations. When K increases from 700 to 1300, due to the increase in solution space, the number of iterations required for four algorithms to converge increases, where LNOHO increases from 41 to 68, LAC increases from 51 to 97, LQBPSO increases from 65 to 115, and LGDS increases from 58 to 101. LNOHO achieves the lowest number of increased iterations compared to LAC, LQBPSO, and LGDS. Moreover, LNOHO achieves the fastest convergence and the lowest objective function value for the same K, followed by LAC, LQBPSO, and LGDS. Table 5 shows the execution time of 150 iterations for solving subproblem **P1** using LNOHO, LAC, LQBPSO, and LGDS. When the number of vehicles is the same, we note that LAC achieves a lower optimum than LQBPSO and LGDS, but LAC has a longer execution time. It is worth noting that LNOHO does not have such a drawback, on the contrary, the execution time of LNOHO is much lower than LAC, LQBPSO and LGDS. Since LAC, LQBPSO and LGDS require a lot of computation in exploring the solution space, the execution time of these algorithms is huge. LNOHO decomposes **P0** into three steps, significantly reduces the execution time due to the low computation complexity of each step. Overall, LNOHO can make higher quality solutions in a shorter time compared to LAC, LQBPSO, and LGDS.

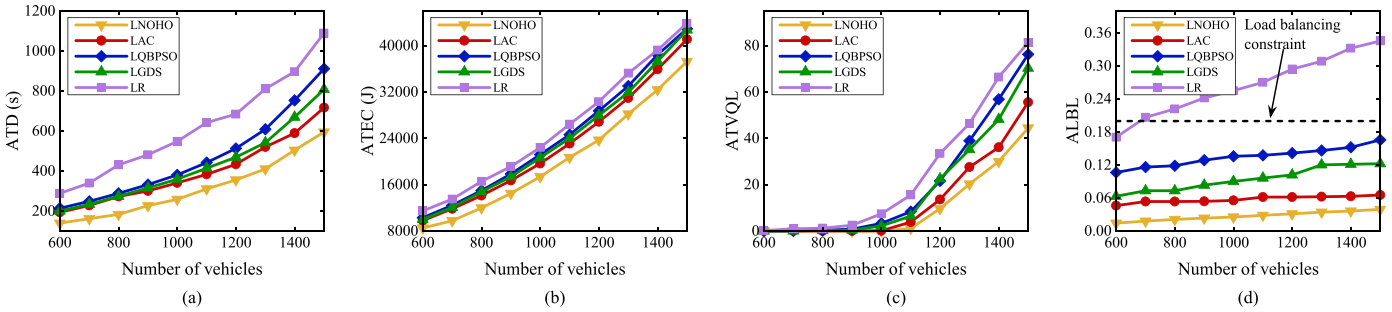


Fig. 6. Trend of different performance indicators over number of vehicles: (a) ATD; (b) ATEC; (c) ATVQL; (d) ALBL.

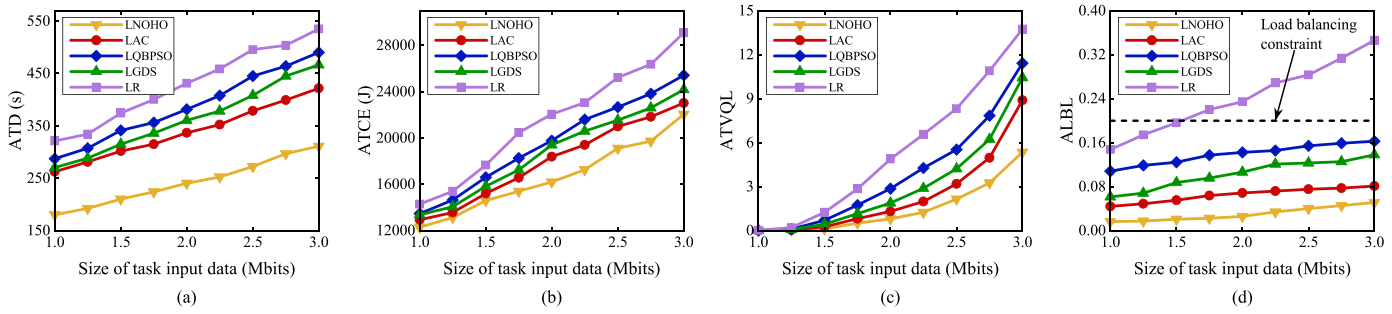


Fig. 7. Trend of different performance indicators over size of task input data: (a) ATD; (b) ATEC; (c) ATVQL; (d) ALBL.

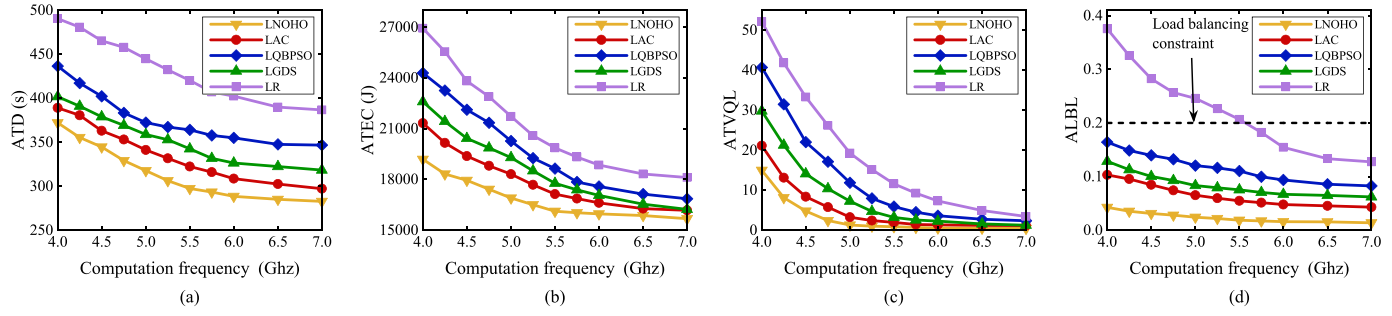


Fig. 8. Trend of different performance indicators over average computation frequency of a virtual machine: (a) ATD; (b) ATEC; (c) ATVQL; (d) ALBL.

By setting number of vehicles $K = 1000$, size of task input data $D_{\text{in}} = 2.5$ Mbits, and average computation frequency of a virtual machine $f_{\text{RSU}} = 5$ GHz as an example, we show the trends of ATD, ATEC, ATVQL, and ALBL over time with different algorithms in Fig. 5. From Figs. 5a and 5b, we observe that five algorithms can achieve convergence in ATD and ATEC, and LNOHO has the lowest convergence values in ATD and ATEC. From Fig. 5c, we see that LNOHO algorithm can achieve the lowest virtual queue value, followed by LAC, LGDS, LQBPSO, and LR, which also means that LNOHO algorithm can better comply with constraint (31a). From Fig. 5d, we can observe that only LNOHO, LAC, LGDS, and LQBPSO can strictly satisfy load balancing constraint (31d) and LNOHO algorithm can achieve the lowest ALBL.

By setting size of task input data $D_{\text{in}} = 2.5$ Mbits, average computation frequency of a virtual machine $f_{\text{RSU}} = 5$ GHz as an example, we show ATD, total energy consumption, ATVQL, and ALBL versus number of vehicles with different algorithms in Fig. 6. Here, each point in the image represents the value of the 150th time slot, i.e., system stability, and five algorithms reach convergence in solving per-slot subproblem **P1**. From Figs. 6a and 6b, we observe that as the number of vehicles increases, ATD and ATEC for five algorithms increase accordingly, and LNOHO achieves the lowest value of ATD and ATEC than other four algorithms. In Fig. 6c, when the number of vehicles is greater than 900, the virtual queues of the five algorithms begin to increase, and LNOHO achieves the lowest ATVQL among other four algorithms. From Fig. 6d, we find that as the number of vehicles increases, ALBL for five algorithms increases accordingly, and all the other four algorithms except LR can guarantee load balancing constraint. For the same number of vehicles, LNOHO achieves the lowest ALBL compared to other algorithms. Since LR is randomly offloading, it fails to satisfy the load balancing constraint when the number of vehicles exceeds 700. Overall analysis shows that LNOHO has the best performance in scenarios with different numbers of vehicles.

By setting number of vehicles $K = 1000$ and average computation frequency of a virtual machine $f_{\text{RSU}} = 5$ GHz as an example, we show ATD, ATEC, ATVQL, and ALBL versus size of task input data with five algorithms in Fig. 7. When size of task input data is small, computation resources of vehicle processor can satisfy the service delay constrain (31c), which leads to smaller ATD, total energy consumption, ATVQL, and ALBL. As size of task input data increases, the vehicle prefers to offload the task to the RSU to ensure that the task can be completed within the allotted time, which leads to an increase in ATD, total energy consumption, ATVQL, and ALBL. Meanwhile, we can find that LNOHO achieves the lowest ATD, ATEC, ATVQL, and ALBL among other four algorithms. Overall analysis shows that LNOHO has the best performance in scenarios with different size of task input data.

By setting number of vehicles $K = 1000$ and size of task input data $D_{\text{in}} = 2.5$ Mbits as an example, we show ATD, ATEC, ATVQL, and ALBL versus average computation frequency of a virtual machine

with different algorithms in Fig. 8. From Figs. 8a and 8b, we find that total delay and ATEC decrease with increasing computation frequency for a virtual machine. However, the decrease is not obvious when computation frequency for a virtual machine is greater than 6 GHz. The increase in computation frequency for a virtual machine leads to an increase in service rate, which decreases processing time and RSU load level. According to Eq. (14), the lower the RSU load level, the lower the average power of the RSU, which leads to a decrease in processing energy consumption. Although computation delay and computation energy consumption can be reduced, other delay and other energy consumption in the offloading process will not be reduced, so the downward trend of ATD and ATEC is not obvious when computation frequency for a virtual machine is higher. Moreover, According to Eqs. (12) and (32), the lower load level of RSU (the higher computation frequency for a virtual machine), the lower ATVQL and ALBL. As shown in Figs. 8c and 8d, we find that as the computation frequency increases, ATVQL and ALBL decrease accordingly close to 0 for LNOHO, LAC, LQBPSO, and LGDS. Meanwhile, we can find that LNOHO achieves the lowest ATD, ATEC, ATVQL, and ALBL among other four algorithms.

6. Conclusions

In this paper, we have studied a MEC-assisted vehicular network architecture that includes I2I, V2I, and I2V links. In the architecture, we have formulated a computation offloading problem with task migration and result feedback to minimize the computation cost. We have designed a vehicle trajectory prediction module based on an encoder–decoder architecture to predict the vehicle position after a task is completed. In order to decouple the decisions under different time slots, we have proposed the LNOHO algorithm, which can transform the continuous multi-slot optimization problem into multiple per-slot deterministic subproblems. In each time slot, the deterministic optimization problem is solved by a load-aware heuristic algorithm in three steps: (1) determine whether a task is processed locally or in the RSU; (2) determine the number of emigration tasks or immigration tasks for RSU; (3) determine decision matrix. The results of traffic simulation based on real scenarios verify the effectiveness of vehicle trajectory prediction module and have also validated the superiority of LNOHO, by comparing it with benchmarks in terms of average total delay, average total energy consumption, average total virtual queue length, and average load balancing level.

If the designed system is deployed in a real road scenario, the hardware needs to meet the following requirements: (1) the vehicle communication module supports the DSRC/5G-CV2X communication protocol; (2) the vehicle server and the edge server should be equipped with high-performance computing units; (3) the edge server is equipped with a high-bandwidth communication interface to support concurrent connections of multiple vehicles. Additionally, the designed system also needs to address the following challenges: (1) how to alleviate communication problems such as signal quality degradation and communication interruption caused by signal occlusion; (2) how to deal with the potential sudden behaviors of vehicle drivers and the uncertainties of road conditions during the trajectory prediction process. To meet the above challenges, in future work, we plan to (1) implement RIS technology to enhance the signal-to-noise ratio by dynamically adjusting signal reflection paths; (2) introduce a Bayesian neural network framework to model the uncertainty of vehicle intentions and explore a road detection algorithm to enhance environmental perception capabilities. Furthermore, we intend to systematically collect parameters of heterogeneous RSU devices and dynamically vary computation capabilities to establish system models that better reflect real-world operational conditions.

CRediT authorship contribution statement

Jingwei Geng: Methodology, Formal analysis, Data curation. **Shunfu Jin:** Validation, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work has been partly supported by National Natural Science Foundation (Grant numbers 62273292 and 62276226), China; Central Leading Local Science and Technology Development Project of Hebei Province (Grant number 246Z0804G), China; Innovation Capability Improvement Plan Project of Hebei Province (Grant number 22567626H), China.

Data availability

No data was used for the research described in the article.

References

[1] S. Niu, R. Dong, W. Liu, P. Ge, Q. Liu, Broadcast signcryption scheme with equality test in smart transportation system, Veh. Commun. 49 (2024) 100820, http://dx.doi.org/10.1016/j.vehcom.2024.100820.
[2] J. Redondo, N. Aslam, J. Zhang, Z. Yuan, Multi-agent assessment with QoS enhancement for HD map updates in a vehicular network and multi-service environment, IEEE Trans. Netw. Sci. Eng. 12 (2) (2025) 738–749, http://dx.doi.org/10.1109/TNSE.2024.3514744.
[3] A. Hazra, D. De, SoftWind: Software-defined trajectory correction modelling of gust wind effects on internet of drone things using glowworm swarm optimization, Ad. Hoc. Netw. 163 (2024) 103577, http://dx.doi.org/10.1016/j.adhoc.2024.103577.
[4] J. Pimentel, The Role of ISO 26262, Society of Automotive Engineers, China, 2020.
[5] H. Wu, Y. Lu, H. Ma, L. Xing, K. Deng, X. Lu, A survey on task type-based computation offloading in mobile edge networks, Ad. Hoc. Netw. 169 (2025) 103754, http://dx.doi.org/10.1016/j.adhoc.2025.103754.
[6] X. He, H. Xu, X. Xu, Y. Chen, Z. Wang, An efficient algorithm for microservice placement in cloud-edge collaborative computing environment, IEEE Trans. Serv. Comput. 17 (5) (2024) 1983–1997, http://dx.doi.org/10.1109/TSC.2024.3399650.
[7] L. Lei, C. Chen, P. Qingqi, M. Sabita, Z. Yan, Vehicular edge computing and networking: A survey, Mob. Netw. Appl. 26 (2021) 1145–1168, http://dx.doi.org/10.1007/s11036-020-01624-1.
[8] S. Jain, V.K. Jain, S. Mishra, Fuzzy-AHP based optimal RSU deployment (fuzzy-AHP-ORD) approach using road and traffic analysis in VANET, Ad. Hoc. Netw. 161 (2024) 103529, http://dx.doi.org/10.1016/j.adhoc.2024.103529.
[9] X. Dai, Z. Xiao, H. Jiang, J.C.S. Lui, UAV-assisted task offloading in vehicular edge computing networks, IEEE Trans. Mob. Comput. 23 (4) (2024) 2520–2534, http://dx.doi.org/10.1109/TMC.2023.3259394.
[10] H. Zhang, L. Feng, X. Liu, K. Long, G.K. Karagiannidis, User scheduling and task offloading in multi-tier computing 6G vehicular network, IEEE J. Sel. Areas Commun. 41 (2) (2023) 446–456, http://dx.doi.org/10.1109/JSAC.2022.3227097.
[11] J. Huang, J. Wan, B. Lv, Q. Ye, Y. Chen, Joint computation offloading and resource allocation for edge-cloud collaboration in internet of vehicles via deep reinforcement learning, IEEE Syst. J. 17 (2) (2023) 2500–2511, http://dx.doi.org/10.1109/JSYST.2023.3249217.
[12] P. Dai, K. Hu, X. Wu, H. Xing, F. Teng, Z. Yu, A probabilistic approach for cooperative computation offloading in MEC-assisted vehicular networks, IEEE Trans. Intell. Transp. Syst. 23 (2) (2022) 899–911, http://dx.doi.org/10.1109/TITS.2020.3017172.
[13] L. Liu, J. Feng, X. Mu, Q. Pei, D. Lan, M. Xiao, Asynchronous deep reinforcement learning for collaborative task computing and on-demand resource allocation in vehicular edge computing, IEEE Trans. Intell. Transp. Syst. (2023) 1–14, http://dx.doi.org/10.1109/TITS.2023.3249745.
[14] Y. Peng, X. Tang, Y. Zhou, J. Li, Y. Qi, L. Liu, H. Lin, Computing and communication cost-aware service migration enabled by transfer reinforcement learning for dynamic vehicular edge computing networks, IEEE Trans. Mob. Comput. 23 (1) (2024) 257–269, http://dx.doi.org/10.1109/TMC.2022.3225239.
[15] S.R. Sahoo, M. Patra, A. Gupta, AALB: Application aware load balancing algorithm for road side units, Veh. Commun. 36 (2022) 100475, http://dx.doi.org/10.1016/j.vehcom.2022.100475.
[16] W. Duan, X. Gu, M. Wen, Y. Ji, J. Ge, G. Zhang, Resource management for intelligent vehicular edge computing networks, IEEE Trans. Intell. Transp. Syst. 23 (7) (2022) 9797–9808, http://dx.doi.org/10.1109/TITS.2021.3114957.

[17] L. Zhao, T. Li, G. Meng, A. Hawbani, G. Min, A.Y. Al-Dubai, A.Y. Zomaya, Novel Lagrange multipliers-driven adaptive offloading for vehicular edge computing, IEEE Trans. Comput. 73 (12) (2024) 2868–2881, http://dx.doi.org/10.1109/TC.2024.3457729.

[18] H. Wang, T. Lv, Z. Lin, J. Zeng, Energy-delay minimization of task migration based on game theory in MEC-assisted vehicular networks, IEEE Trans. Veh. Technol. 71 (8) (2022) 8175–8188, http://dx.doi.org/10.1109/TVT.2022.3175238.

[19] Y. Sun, X. Guo, J. Song, S. Zhou, Z. Jiang, X. Liu, Z. Niu, Adaptive learning-based task offloading for vehicular edge computing systems, IEEE Trans. Veh. Technol. 68 (4) (2019) 3061–3074, http://dx.doi.org/10.1109/TVT.2019.2895593.

[20] L. Chen, J. Xu, Task replication for vehicular cloud: Contextual combinatorial bandit with delayed feedback, in: IEEE INFOCOM 2019 - IEEE Conference on Computer Communications, 2019, pp. 748–756, http://dx.doi.org/10.1109/INFOCOM.2019.8737654.

[21] Z. Nan, S. Zhou, Y. Jia, Z. Niu, Joint task offloading and resource allocation for vehicular edge computing with result feedback delay, IEEE Trans. Wirel. Commun. 22 (10) (2023) 6547–6561, http://dx.doi.org/10.1109/TWC.2023.3244391.

[22] Z. Ding, J. Xu, O.A. Dobre, H.V. Poor, Joint power and time allocation for NOMA–MEC offloading, IEEE Trans. Veh. Technol. 68 (6) (2019) 6207–6211, http://dx.doi.org/10.1109/TVT.2019.2907253.

[23] M. Yan, R. Xiong, Y. Wang, C. Li, Edge computing task offloading optimization for a UAV-assisted internet of vehicles via deep reinforcement learning, IEEE Trans. Veh. Technol. 73 (4) (2024) 5647–5658, http://dx.doi.org/10.1109/TVT.2023.3331363.

[24] F.G. Wakgra, B. Kar, S.B. Tadele, S.-H. Shen, A.U. Khan, Multi-objective offloading optimization in MEC and vehicular-fog systems: A distributed-TD3 approach, IEEE Trans. Intell. Transp. Syst. (2024) 1–13, http://dx.doi.org/10.1109/TITS.2024.3409367.

[25] K. Fang, Y. Ouyang, B. Zheng, L. Huang, G. Wang, Z. Chen, Security enhancement for RIS-Aided MEC systems with deep reinforcement learning, IEEE Trans. Commun. 73 (4) (2025) 2466–2479, http://dx.doi.org/10.1109/TCOMM.2024.3476139.

[26] Y. Cui, H. Li, D. Zhang, A. Zhu, Y. Li, H. Qiang, Multiagent reinforcement learning-based cooperative multitype task offloading strategy for Internet of vehicles in B5G/6G network, IEEE Internet Things J. 10 (14) (2023) 12248–12260, http://dx.doi.org/10.1109/JIOT.2023.3245721.

[27] Y. Hou, Z. Wei, R. Zhang, X. Cheng, L. Yang, Hierarchical task offloading for vehicular fog computing based on multi-agent deep reinforcement learning, IEEE Trans. Wirel. Commun. 23 (4) (2024) 3074–3085, http://dx.doi.org/10.1109/TWC.2023.3305321.

[28] K. Elham, C. Yuanzhu, A. Behzad, Task offloading in vehicular edge computing networks via deep reinforcement learning, Comput. Commun. 189 (2022) 193–204, http://dx.doi.org/10.1016/j.comcom.2022.04.006.

[29] H. Wu, J. Chen, W. Xu, N. Cheng, W. Shi, L. Wang, X. Shen, Delay-minimized edge caching in heterogeneous vehicular networks: A matching-based approach, IEEE Trans. Wirel. Commun. 19 (10) (2020) 6409–6424, http://dx.doi.org/10.1109/TWC.2020.3003339.

[30] M. Liwang, S. Dai, Z. Gao, Y. Tang, H. Dai, A truthful reverse-auction mechanism for computation offloading in cloud-enabled vehicular network, IEEE Internet Things J. 6 (3) (2019) 4214–4227, http://dx.doi.org/10.1109/JIOT.2018.2875507.

[31] F. Sun, F. Hou, N. Cheng, M. Wang, H. Zhou, L. Gui, X. Shen, Cooperative task scheduling for computation offloading in vehicular cloud, IEEE Trans. Veh. Technol. 67 (11) (2018) 11049–11061, http://dx.doi.org/10.1109/TVT.2018.2868013.

[32] T.R. Kuo, D. Li, GBHO: A gain-based heuristic offloading algorithm in vehicular edge computing, in: 2022 IEEE 95th Vehicular Technology Conference: (VTC2022-Spring), 2022, pp. 1–7, http://dx.doi.org/10.1109/VTC2022-Spring54318.2022.9860706.

[33] C. Tang, C. Zhu, H. Wu, Q. Li, J.J.P.C. Rodrigues, Toward response time minimization considering energy consumption in caching-assisted vehicular edge computing, IEEE Internet Things J. 9 (7) (2022) 5051–5064, http://dx.doi.org/10.1109/JIOT.2021.3108902.

[34] M. Yan, R. Xiong, Y. Wang, C. Li, Edge computing task offloading optimization for a UAV-assisted Internet of vehicles via deep reinforcement learning, IEEE Trans. Veh. Technol. 73 (4) (2024) 5647–5658, http://dx.doi.org/10.1109/TVT.2023.3331363.

[35] H. Huang, W. Zhan, G. Min, Z. Duan, K. Peng, Mobility-aware computation offloading with load balancing in smart city networks using MEC federation, IEEE Trans. Mob. Comput. 23 (11) (2024) 10411–10428, http://dx.doi.org/10.1109/TMC.2024.3376377.

[36] S. Ross, Introduction to Probability Models, Academic Press, USA, 2014.

[37] M. Portnoy, Virtualization Essentials, Wiley, USA, 2012.

[38] L. Codeca, R. Frank, S. Faye, T. Engel, Luxembourg SUMO traffic (LuST) scenario: Traffic demand evaluation, IEEE Intell. Transp. Syst. Mag. 9 (2) (2017) 52–63, http://dx.doi.org/10.1109/MITS.2017.2666585.

[39] H. Guo, L. Rui, Z. Gao, V2V task offloading algorithm with LSTM-based spatiotemporal trajectory prediction model in SVCNs, IEEE Trans. Veh. Technol. 71 (10) (2022) 11017–11032.

[40] A. Abdelraouf, R. Gupta, K. Han, Interaction-aware personalized vehicle trajectory prediction using temporal graph neural networks, in: 2023 IEEE 26th International Conference on Intelligent Transportation Systems, ITSC, 2023, pp. 2070–2077, http://dx.doi.org/10.1109/ITSC57777.2023.10421928.

[41] L. Lin, W. Li, H. Bi, L. Qin, Vehicle trajectory prediction using LSTMs with spatial–temporal attention mechanisms, IEEE Intell. Transp. Syst. Mag. 14 (2) (2022) 197–208, http://dx.doi.org/10.1109/MITS.2021.3049404.

[42] A. Fréville, The multidimensional 0–1 knapsack problem: An overview, European J. Oper. Res. 155 (1) (2004) 1–21, http://dx.doi.org/10.1016/S0377-2217(03)00274-1.

[43] V. Boyer, M. Elkihel, D. El Baz, Heuristics for the 0–1 multidimensional knapsack problem, European J. Oper. Res. 199 (3) (2009) 658–664, http://dx.doi.org/10.1016/j.ejor.2007.06.068.

[44] Y. Ju, Y. Chen, Z. Cao, L. Liu, Q. Pei, M. Xiao, K. Ota, M. Dong, V.C.M. Leung, Joint secure offloading and resource allocation for vehicular edge computing network: A multi-agent deep reinforcement learning approach, IEEE Trans. Intell. Transp. Syst. 24 (5) (2023) 5555–5569, http://dx.doi.org/10.1109/TITS.2023.3242997.

[45] Y. Lin, Y. Zhang, J. Li, F. Shu, C. Li, Popularity-aware online task offloading for heterogeneous vehicular edge computing using contextual clustering of bandits, IEEE Internet Things J. 9 (7) (2022) 5422–5433, http://dx.doi.org/10.1109/JIOT.2021.3109003.

[46] X. Liu, J. Zheng, Y. Li, M. Zhang, R. Wang, Y. He, Multi-path serial tasks offloading strategy and dynamic scheduling optimization in vehicular edge computing networks, Veh. Commun. 49 (2024) 100827, http://dx.doi.org/10.1016/j.vehcom.2024.100827.

[47] Y. Sun, Z. Wu, K. Meng, Y. Zheng, Vehicular task offloading and job scheduling method based on cloud-edge computing, IEEE Trans. Intell. Transp. Syst. 24 (12) (2023) 14651–14662, http://dx.doi.org/10.1109/TITS.2023.3300437.

[48] L. Chen, D. Zhang, J. Zhang, T. Zhang, W.-J. Wang, Y. Cao, A novel offloading approach of IoT user perception task based on quantum behavior particle swarm optimization, Future Gener. Comput. Syst. 141 (2023) 577–594, http://dx.doi.org/10.1016/j.future.2022.12.016.

[49] E. Rajapackiyam, A. Devi, M.S. Reddy, U. Arumugam, S. Vairavasundaram, I. Vairavasundaram, V. Suresh, An efficient computation offloading in edge environment using genetic algorithm with directed search techniques for IoT applications, Future Gener. Comput. Syst. 158 (2024) 378–390, http://dx.doi.org/10.1016/j.future.2024.04.021.